

LearnCpp Chapter 2

C++ Basics: Functions & Files

Field notes, rewritten from scratch • companion to learncpp.com Ch. 2 • July 2026

How these notes work (read this once)

One chapter, one operating system. Every section below has the exact same skeleton, so your brain never has to ask “what am I looking at?” — it only has to ask “what does this say?”

1. **Meta line** — estimated minutes + what it builds on. Pick sections that fit the time you actually have.
2. **IN ONE BREATH** — the whole section in one sentence. If you remember nothing else, remember this line.
3. **Short chunks** — never a wall of text. Prose, then a box, then code, then prose.
4. **QUICK CHECK** — 30–60 seconds of active recall. *Do not skip it.* Answer out loud or on paper, then check the back. If you miss it, re-read only that section, not the chapter.
5. **Tick the map** — check the box on the progress map. Closure is fuel.

The seven box types (color = meaning, always)

■	DEFINITION	Precise vocabulary. These words have exact meanings — use them exactly.
■	KEY IDEA	The mental model. <i>Why</i> this thing exists, usually with an analogy.
■	UNDER THE HOOD	What the machine actually does: memory, compiler, linker.
■	GOTCHA	A real bug, always with When it bites — the exact phase and trigger.
■	PRECISION	Two terms people blur together, pulled apart. Say the right one.
■	BRIDGE	Hooks this idea to something you already own (projects, prior concepts).
■	QUICK CHECK	Active recall. The learning happens here, not in the reading.

Session protocol (30/60 rhythm)

30 minutes reading $\approx$ two to three sections, Quick Checks included. **60 minutes doing** = drills from the bank at the back (they cite the sections they test) or the build tasks. Whole chapter $\approx$ three hours of reading spread over as many sittings as you want. If you drift mid-section: name it, tick nothing, come back to the *IN ONE BREATH* line and restart from there.

Progress map

	§	Lesson	min	The one thing to walk away with
☐	2.1	Introduction to functions	10	A function is a named, reusable job; caller vs. callee.
☐	2.2	Function return values	15	<code>return</code> copies a value back; missing <code>return</code> = UB.
☐	2.3	Void functions	8	<code>void</code> = does a job, hands nothing back.
☐	2.4	Parameters and arguments	12	Argument is copied into parameter (pass by value).
☐	2.5	Local scope	15	Scope is compile-time; lifetime is runtime. Stack frames.
☐	2.6	Why functions are useful	10	When and why to extract a function.
☐	2.7	Forward declarations, ODR	20	Declaration = promise to compiler; definition pays the linker.
☐	2.8	Multiple code files	15	Each <code>.cpp</code> compiles alone; the linker stitches.
☐	2.9	Naming collisions, namespaces	12	Namespaces are surnames; <code>::</code> is addressing.
☐	2.10	The preprocessor	15	A text machine runs before the compiler.
☐	2.11	Header files	20	A header is the published interface of a <code>.cpp</code> .
☐	2.12	Header guards	12	Guards stop double-paste <i>within one</i> translation unit.
☐	2.13	Designing your first programs	15	Decompose top-down before typing anything.

After 2.13: the Big Picture page (build pipeline + error triage), the one-page cheat sheet, and the 15-drill bank.

2.1 Introduction to functions

TIME 10 min • BUILDS ON Ch. 1 (statements, `main()`)

IN ONE BREATH *A function is a named, reusable sequence of statements that does one job; when it runs, control jumps in, does the job, and jumps back.*

DEFINITION • The cast of characters

A **function** is a reusable sequence of statements designed to do a particular job. The function that starts a call is the **caller**; the function being run is the **callee**. A **function call** is the expression that transfers control to the callee (don't forget the parentheses: `doThing()` ;). Functions you write are **user-defined functions**; the ones shipped with C++ live in the **standard library**. The braces and statements form the **function body**.

Minimal shape. For now every function you write looks like this — a return type, a name, empty parentheses, a body:

```
void sayHi() // header: tells the compiler this function exists
{
    // body: what the function actually does
    std::cout << "Hi\n";
}

int main()
{
    sayHi(); // main is the caller, sayHi is the callee
    sayHi(); // reusable: call it as many times as you like
    return 0;
}
```

Execution order. When `main` hits `sayHi()` ; , `main` *pauses*. Control jumps to the top of `sayHi`, runs its body to the end, then jumps back to the statement right after the call, and `main` resumes. Calls can nest: `main` calls `a`, `a` calls `b`, and each finished callee returns control to whoever called it.

KEY IDEA • Functions are the unit of decomposition

Think of a professional kitchen. `main()` is the head chef: it doesn't chop, sear, and plate everything itself — it delegates to stations that each do one nameable job and hand back a result. Programs scale the same way: `main` should read like the chef calling out the order of operations, not like one person doing every task inline.

GOTCHA • Nested function definitions are illegal

You cannot *define* a function inside another function. (Calling one inside another is the whole point; defining one inside another is banned.) **When it bites:** compile time, immediately, in that file — the compiler rejects the inner definition. Fix: move the inner function outside, above or beside its caller.

QUICK CHECK • 2.1 • answers at the back

Q1. In `int main() { greet(); }` — which function is the caller and which is the callee? **Q2.** Is this legal? `void outer() { void inner() {} }` **Q3.** What happens to `main`'s execution while `greet()` runs?

2.2 Function return values

TIME 15 min • BUILDS ON 2.1

IN ONE BREATH *A value-returning function promises its caller a value of a stated type; `return` fulfills the promise by copying that value back — and breaking the promise is undefined behavior, not a polite error.*

DEFINITION • Return machinery

The **return type** (before the name) declares what type of value the function sends back. A **return statement** (`return expr;`) ends the function and produces the **return value**. The value is *copied* back to the caller — this is **return by value**. A function whose return type is not `void` is a **value-returning function**, and it *must* return a value on every path.

```
int getSquare(int) ; // ignore parameters until 2.4 -- focus on the int out front

int five()
{
    return 5;          // copy 5 back to whoever called five()
    // anything after a return never runs ("dead code")
}

int main()
{
    int x { five() };    // the call five() *is* an expression with value 5
    std::cout << five() + 2; // return values compose into bigger expressions
    return 0;
}
```

The call is an expression. `five()` evaluates to 5. That’s why you can initialize with it, print it, or feed it into arithmetic. A function returns exactly one value per call (for now — there are richer options much later).

KEY IDEA • Return by value = handing back a photocopy

The callee fills out a form and hands the caller a *photocopy*; the callee’s own copy is destroyed when the function ends (its locals die — see 2.5). This is why there’s never a question of “who owns the returned value”: the caller receives a fresh, independent copy.

UNDER THE HOOD • Where the value physically travels

By common convention, small return values ride back to the caller in a CPU register (on x86-64, integer returns typically use `RAX`); large objects go through memory the caller sets aside. You don’t control this yet — but it explains the gotcha below: if you forget to `return`, the caller still reads that register, and it contains *whatever was left there*.

GOTCHA • Falling off the end of a non-void function is UB

```
int sign(int x)
{
    if (x > 0)
        return 1;
}
```

```
} // x <= 0 ? no return executes. Undefined behavior.
```

When it bites: it *compiles* (usually just a warning). At runtime the caller receives garbage — often “works” in a debug build, then returns nonsense under `-O2` or on a different compiler. Nothing crashes on cue; the failure is silent and unstable. Cover every path, and heed the warning. (Exception: `main` — see below.)

PRECISION • Undefined behavior vs. runtime error

A **runtime error** is *defined* failure: the language or library specifies what happens (e.g., an exception is thrown). **Undefined behavior (UB)** means the C++ standard places *no requirements at all* on what happens — crash, garbage, or *appearing to work* are all conforming outcomes. “It ran fine” is never evidence that UB is absent. Missing `return` = UB, not a runtime error.

main is special. Its return value is the program’s **status code**: 0 (or `EXIT_SUCCESS` from `<cstdlib>`) means success; `EXIT_FAILURE` means failure — the operating system and scripts read this. `main` is the *only* function allowed to omit its return (it implicitly returns 0), and you are not allowed to call `main` yourself.

DRY — Don’t Repeat Yourself. If the same logic appears twice, extract it into a function and call it twice. One home for the logic means one place to fix, test, and change. (Its evil twin has a name too: `DRP` — don’t repeat the same bug in five places.)

QUICK CHECK • 2.2 • answers at the back

Q1. `int f() { if (ready) return 1; }` — what happens when `ready` is false, and *when* do you find out? **Q2.** Why is the second statement here dead? `return 5; return 7;` **Q3.** What two things make `main`’s return special?

2.3 Void functions (non-value returning)

TIME 8 min • BUILDS ON 2.2

IN ONE BREATH A *void function* does a job and hands nothing back — call it for its effect, never for a value.

DEFINITION • Void and early return

A **non-value returning function** declares return type `void`: it returns to the caller but sends no value. A bare `return;` inside it is an **early return** — exit now, skip the rest. Control also returns automatically at the closing brace (no return statement required).

```
void printGreeting()
{
    std::cout << "Welcome\n";
    return; // optional and redundant here -- the brace suffices
}

int main()
{
    printGreeting(); // fine: call as a standalone statement
    // std::cout << printGreeting(); // COMPILE ERROR: no value to print
}
```

```
    return 0;
}
```

Two compile-time rules. (1) A `void` function that tries to `return someValue;` won't compile. (2) Using a `void` call where a value is required — printing it, assigning it, adding it — won't compile. Both are caught early and loudly, which makes them the *friendly* kind of error.

KEY IDEA • Commands vs. queries

Value-returning functions are *queries* (“compute and tell me”); `void` functions are *commands* (“go do this side effect” — print, mutate state). Keeping the two roles distinct keeps interfaces honest: a caller can see from the signature alone whether calling twice is harmless or does the thing twice.

BRIDGE • Your order validator already chose sides

Your validator returns an `enum class result` instead of printing an error itself — that's choosing *query* over *command*, which is exactly what let the I/O stay in the app layer. Chapter 2 is giving you the vocabulary for a decision you already made.

QUICK CHECK • 2.3 • answers at the back

Q1. Predict both problems: `void beep() { return 1; }` and `int x { beep() };` — what kind of error, and when? **Q2.** When is a bare `return;` in a `void` function actually useful (vs. redundant)?

2.4 Introduction to function parameters and arguments

TIME 12 min • BUILDS ON 2.2, 2.3

IN ONE BREATH *Parameters are the variables in a function's signature; arguments are the values the caller supplies; at each call, every argument is copied into its parameter — that copying is pass by value.*

DEFINITION • Inputs, both ends

A **function parameter** is a variable declared in the function header, initialized fresh at every call. An **argument** is the value the caller passes at the call site. Argument → copied into → parameter: this mechanism is **pass by value**. An **unnamed (unreferenced) parameter** is a parameter given a type but no name — legal, used when the value isn't needed but the signature must keep its shape.

```
int add(int x, int y)    // x, y : parameters (live inside add)
{
    return x + y;
}

int main()
{
    int a { 4 };
    std::cout << add(a, 5);    // a and 5 : arguments (live at the call site)
    std::cout << add(1 + 2, 3*3); // any expression works -- it's evaluated,
    return 0;                // then its value is copied in
}
```

PRECISION • Parameter vs. argument

Parameter = the named slot, defined once in the function header, exists inside the callee. **Argument** = the value plugged into that slot, written at each call, belongs to the caller's world. One function, one set of parameters; a thousand calls, a thousand sets of arguments. Interviewers and compiler messages both use these precisely — so should you.

UNDER THE HOOD • The copy is physically real

At the call, each argument's value is copied into fresh storage in the callee's stack frame (see 2.5). Separate bytes, separate lifetime. Two consequences: (1) the callee can scribble on its parameters freely — the caller's variables can't feel it; (2) copying has a *cost*, invisible for an `int`, very visible for a large object. That cost is why `const` references exist — already on your study queue; this is the problem they solve.

GOTCHA • Mutating a parameter and expecting the caller to notice

```
void doubleIt(int x) { x = x * 2; } // doubles the copy...

int main()
{
    int n { 10 };
    doubleIt(n);
    std::cout << n; // prints 10, not 20
}
```

When it bites: runtime, silently — it compiles clean, runs clean, and simply does nothing to `n`. Pure logic bug; no diagnostic will ever point at it. If the caller needs the result, return it: `n = doubleIt(n);`.

Composition. Because calls are expressions (2.2), return values can feed straight into arguments: `print(add(4, 5));` — inner call finishes first, its value becomes the outer call's argument.

QUICK CHECK • 2.4 • answers at the back

Q1. In `multiply(width, 3)` calling `int multiply(int a, int b)` — name every parameter and every argument. **Q2.** Why does the `doubleIt` bug produce no compiler diagnostic, ever? **Q3.** What executes first in `f(g(1), 2)`?

2.5 Introduction to local scope

TIME 15 min • BUILDS ON 2.4

IN ONE BREATH *Scope is a compile-time question — where may this name be used? Lifetime is a runtime question — when does this object exist? For locals the two usually coincide, but they are different axes.*

DEFINITION • Locals, scope, lifetime

Variables defined inside a function body — including parameters — are **local variables**. A local's **lifetime** runs from its point of definition to the end of its enclosing `{ }` block, where it is destroyed.

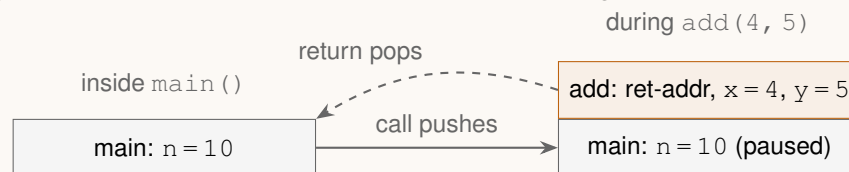
An identifier's **scope** is the region of source code where the name is usable; locals have **block scope**. Using a name outside its scope is a compile error; a variable no longer alive has *gone out of existence*, and touching such an object (via tricks you'll meet later) is UB.

PRECISION • Scope vs. lifetime — different axes

Scope is a property of *names*, settled entirely at *compile time*: can this line legally mention *x*? **Lifetime** is a property of *objects*, played out at *runtime*: do *x*'s bytes currently hold a live object? They usually travel together for locals — but you already own the counterexample: a `static` local (your earlier deep-dive) has *tiny scope* (one function) yet *whole-program lifetime*. Two axes, not one.

UNDER THE HOOD • Stack frames — where locals physically live

Each function call pushes a **stack frame**: a block of memory holding the return address, the parameters (freshly copied — 2.4), and the function's locals. Returning *pops* the frame.



Precision matters here: popping **moves the stack pointer**; it does not wipe the bytes. The old values may linger in memory until something overwrites them — which is exactly why reading a dead object is UB that *sometimes appears to work*. Say “the frame was popped,” not “the memory was wiped.”

Independence. Each call gets its own frame, so each function's locals are private to that call. Two functions can both define `int x` — two names, two objects, zero interaction. Even two *calls* of the same function each get fresh locals.

Best practice. Define locals as close to their first use as reasonable, in the tightest block that needs them. Small scopes = fewer names alive at once = fewer ways to grab the wrong one.

QUICK CHECK • 2.5 • answers at the back

Q1. `void f() { int x{1}; } void g() { int x{2}; }` — same object? Same name legality? Explain using both axes. **Q2.** When `add` returns, what happens to the memory that held `x` and `y` — wiped, or something more precise? **Q3.** Which is checked at compile time: scope violations or lifetime violations?

2.6 Why functions are useful, and how to use them effectively

TIME 10 min • BUILDS ON 2.1–2.5

IN ONE BREATH *Functions buy you five things — organization, reuse, testing, extensibility, abstraction — and the skill is knowing when a block of code has earned its own name.*

The five payoffs

- **Organization** — `main` shrinks to a readable table of contents; complexity hides behind names.
- **Reusability (DRY)** — one implementation, many call sites; fix a bug once, it's fixed everywhere.

- **Testing** — a small function with clear inputs/outputs can be tested in isolation; once proven, you stop re-testing it every time you use it. Fewer lines of *unique* code = smaller surface for bugs.
- **Extensibility** — new behavior lands in one place, and every caller inherits it.
- **Abstraction** — callers need the *name, inputs, and output* — never the internals. You use `std::cout` daily without reading its implementation; your functions grant others the same privilege.

KEY IDEA • A function is a contract with a front panel

A vending machine: coins in, snack out, mechanism invisible. That front panel *is* abstraction — and it's also why refactoring is safe: you can gut the machine's insides, and as long as coins-in/snack-out holds, no caller can tell. **Refactoring** = restructuring code without changing its observable behavior.

When does code earn its own function?

Extract a function when a block: appears (or will appear) **more than once**; forms a **distinct sub-task** with a natural name and a clean input/output boundary; or makes its host function **too long to read at a glance**. Corollary: a function should do *one job* — if you can only name it `processAndPrintAndSave`, that's three functions wearing a trench coat.

BRIDGE • This is “make it right”

“Make it work, make it right, make it fast” — extraction and naming are core *make-it-right* moves. And notice the layering you already practice: every function is a tiny library to its caller. `main` is just the app layer of your own functions.

QUICK CHECK • 2.6 • answers at the back

Q1. Name the five payoffs without looking. **Q2.** `main` reads input, validates it, computes a fee, prints a report — 40 lines, each step once. No repetition, so does DRY apply? Which payoffs still argue for extraction?

2.7 Forward declarations and definitions

TIME 20 min • BUILDS ON 2.1–2.6 • the pivotal section of the chapter

IN ONE BREATH *The compiler reads a file top-to-bottom with no lookahead, so a call must be preceded by at least a declaration — a promise that the function exists; the definition is what later pays that promise off, and the linker is the collector.*

The problem: order matters

```
int main()
{
    std::cout << add(4, 5); // ERROR: 'add' was not declared in this scope
    return 0;
}
int add(int x, int y) { return x + y; } // too late -- compiler already gave up
```

The compiler compiles **sequentially**: at the call site it has never heard of `add`, so it can't type-check the call. Two fixes: **(1) reorder** — define `add` above `main` (fine in small files, collapses in big ones and can't fix mutual recursion); **(2) forward declare**.

DEFINITION • Declaration vs. definition

A **declaration** tells the compiler an identifier exists and what type it has — enough to type-check uses. A **definition** actually *implements* the function (or type) or *instantiates* the variable — it's what the linker ultimately needs. A **function prototype** is the declaration form for functions: header, no body — `int add(int x, int y);` — parameter names optional: `int add(int, int);`. **Every definition is also a declaration**; the reverse is false. Declarations that aren't definitions are **pure declarations**. A **forward declaration** is declaring an identifier before its definition appears.

```
int add(int x, int y);           // forward declaration: a promise

int main()
{
    std::cout << add(4, 5); // compiler: "type-checks against the promise. fine."
    return 0;
}

int add(int x, int y)           // definition: the promise, paid
{
    return x + y;
}
```

KEY IDEA • IOUs and the bank

A declaration is an **IOU note**: the compiler happily accepts it and keeps compiling. The **linker is the bank**: at the end of the build, every IOU that was actually used must be backed by *exactly one* real definition somewhere in the program. Promise never paid → the bank bounces the build.

The One Definition Rule (ODR)

- Part 1. Within one file (translation unit):** each function, variable, type, or template may be *defined* at most once. Violation → **compile-time** redefinition error.
- Part 2. Across the whole program:** each function or variable that is actually used must have *exactly one* definition. Zero → linker “undefined reference”; two-plus → linker “multiple definition.”
- Part 3. The escape hatch:** types, templates, and `inline` functions/variables *may* be defined in multiple files — provided the definitions are token-for-token identical. (Park this; it becomes crucial when headers start carrying class definitions.)

GOTCHA • Declared, used, never defined

Forward declare `add`, call it, never write the body. **When it bites: link time**, not compile time — and *only if the call actually exists*. Each `.cpp` compiles clean (the promise satisfied the compiler); the final link step fails with *undefined reference to 'add'*. Late, and phrased in linker dialect — which is why people find it confusing. You won't: no definition anywhere = linker's complaint.

PRECISION • “Doesn't compile” vs. “doesn't link”

Two different collectors, two different debts. **Compile error** = this one file, on its own, is ill-formed (unknown name, bad types, redefinition within the file). **Link error** = every file compiled fine, but the program-wide books don't balance (missing or duplicate definitions). Diagnosing starts with asking *which phase* complained — see the triage table after 2.13.

QUICK CHECK • 2.7 • answers at the back

Q1. Classify each: `int add(int, int);` / `int add(int x, int y) { return x+y; }` / `int x;` — declaration, definition, or both? **Q2.** Forward-declared, called, no body anywhere: which phase errors, and what's the message's key phrase? **Q3.** Same function defined twice *in one file*: which ODR part, which phase?

2.8 Programs with multiple code files

TIME 15 min • BUILDS ON 2.7

IN ONE BREATH Each `.cpp` file is compiled alone into an object file — files never see each other — and the linker then stitches the object files into one executable, resolving every promise against every payment.

DEFINITION • Build vocabulary

A **translation unit (TU)** is one `.cpp` after preprocessing (2.10) — the actual unit the compiler consumes. Compiling a TU yields an **object file** (`.o` / `.obj`): machine code plus a ledger of symbols it *defines* and symbols it *needs*. **Linking** merges object files, matching every “needs” to exactly one “defines,” producing the executable.

The consequence. When the compiler works on `main.cpp`, it has *zero knowledge* of `add.cpp` — not “limited,” zero. So if `main.cpp` calls `add`, `main.cpp` itself must contain a declaration of `add`. For now, hand-write the forward declaration; in 2.11 a header will deliver it properly.

```
// add.cpp
int add(int x, int y) { return x + y; }           // the definition lives here

// main.cpp
#include <iostream>
int add(int x, int y);                           // promise, local to this TU
int main() { std::cout << add(4, 5); return 0; }
```

UNDER THE HOOD • Why builds scale — and why yours rebuilds fast

Because TUs compile independently, editing `add.cpp` requires recompiling *only* `add.o`, then relinking. That is the entire basis of incremental builds. Your `new_cpp` scaffold rides this: CMake records which sources make the target, Ninja compiles each TU (in parallel) and runs one link step — touch one file, and Ninja redoes exactly one compile plus the link.

GOTCHA • Two failure modes that look similar and aren't

(a) “*add was not declared in this scope*” → **compile time**: this TU lacks a declaration. Fix inside the file (declare/include). (b) “*undefined reference to add*” → **link time**: declaration fine, but no linked object file defines it — classic cause: `add.cpp` was never added to the build. **When each bites**: (a) while compiling that one file; (b) only at the final link, after every file compiled “successfully.”

QUICK CHECK • 2.8 • answers at the back

Q1. Why must `main.cpp` contain its own declaration of `add` even though `add.cpp` sits right next to it? **Q2.** You created `helper.cpp` but forgot to list it in `CMakeLists.txt`. Which error, which phase? **Q3.** What two ledgers does an object file carry?

2.9 Naming collisions and an introduction to namespaces

TIME 12 min • BUILDS ON 2.8

IN ONE BREATH *When two identifiers share a name in a way the compiler or linker can't disambiguate, the build errors; namespaces fix this by giving names a surname, addressed with `::`.*

Collision, two flavors. Define two functions named `doThing` (same signature): **in the same TU** → compiler redefinition error (ODR part 1). **In two different TUs of one program** → each file compiles happily; the *linker* finds two payments for one name and errors (ODR part 2). More files, more people, more collisions — this is a scaling problem, and it needs a scaling fix.

DEFINITION • Namespace and the scope-resolution operator

A **namespace** is a named region that guarantees identifiers inside it are unique *within it*; the same bare name may exist in different namespaces without conflict. The **scope-resolution operator** `::` addresses into one: `std::cout` means “the `cout` that lives in namespace `std`” — the namespace the entire C++ standard library resides in.

KEY IDEA • Surnames and area codes

Two Rohans in one classroom is chaos; *Rohan Mehta* vs. *Rohan Iyer* is a Tuesday. Namespaces are surnames for identifiers; `::` is how you say the full name. Same reason phone systems have area codes: local names stay short, global names stay unique.

GOTCHA • `using namespace std;` — convenience with a fuse

It imports *every* `std` name into your scope, so any of your identifiers may now collide with any current — or *future* — standard-library name (write your own `count` or `max` and enjoy the ambiguity). **When it bites:** compile time, but possibly *years later* — your code breaks when a new compiler/library version adds a name you'd been using, with errors pointing somewhere unhelpful. Prefer explicit `std::`; the five keystrokes buy determinism. (In *headers* it's outright hazardous — you'd force the import on every includer; see 2.11.)

QUICK CHECK • 2.9 • answers at the back

Q1. Duplicate definition in one TU vs. across two TUs — which tool reports each, and at which phase? **Q2.** What precisely does `std::cout` say, word by word? **Q3.** Why can `using namespace std;` break code that compiled fine for years?

2.10 Introduction to the preprocessor

TIME 15 min • BUILDS ON 2.8

IN ONE BREATH Before the compiler sees a single token, a pure text machine runs over the file — pasting `#includes`, expanding `#defines`, and evaluating conditional-compilation — and its output, the translation unit, is what actually gets compiled.

DEFINITION • Preprocessor and directives

The **preprocessor** is a stage that transforms the *text* of your source before compilation (it also strips comments). Its instructions are **directives**: lines starting with `#`, ending at the newline — no semicolons. Its output per file is the **translation unit**. The preprocessor knows nothing about C++ semantics; it manipulates text.

The three directives that matter now

1) **#include** — literally replaces the directive with the full text of the named file, then preprocesses *that* too. After preprocessing, there is no “include” — just one long pasted document. This is the mechanism headers ride on (2.11).

2) **#define (object-like macros)** — token find-and-replace from this point down:

```
#define PI 3.14159           // with substitution text: every later PI -> 3.14159
#define ENABLE_LOGS         // without substitution text: PI-style replacement by
                             // *nothing*; the name now simply "is defined"
```

The no-substitution form exists purely as a flag for conditional compilation. The with-substitution form is **legacy style** — modern C++ prefers `constexpr` constants (coming in Ch. 5), because macros ignore both scope and the type system.

3) **Conditional compilation** — include or delete regions of text before the compiler ever sees them:

```
#ifdef ENABLE_LOGS
    std::cout << "log line\n";    // kept only if ENABLE_LOGS is defined
#endif
#ifndef ENABLE_LOGS
    // kept only if it is NOT defined
#endif
#if 0
    ...disabled block: handy for "commenting out" code that
    already contains /* block comments */, which cannot nest...
#endif
```

KEY IDEA • An intern with scissors, glue, and a photocopier

The preprocessor is an intern editing a *photocopy* of your file: paste this file here, replace that word everywhere below, cut this region. The intern cannot read C++ — only text. Every “weird” preprocessor behavior becomes obvious once you remember it is text-in, text-out, semantics-blind.

GOTCHA • Directive reach: this file, top to bottom, and no further

A `#define` takes effect from its line to the *end of that translation unit* — ignoring functions, blocks, and namespaces entirely (text, remember) — and it does *not* leak into other .cpp files, because each TU is preprocessed independently. **When it bites:** (a) a macro defined mid-file silently rewrites identical tokens *anywhere below*, even inside unrelated functions; (b) a flag defined in a .cpp does

nothing for `b.cpp` — “but I defined it!” — wrong TU.

QUICK CHECK • 2.10 • answers at the back

Q1. After preprocessing, what remains of the line `#include <iostream>`? **Q2.** `#define MAX 100` sits in `a.cpp`. Does `MAX` mean anything in `b.cpp`? Why, mechanically? **Q3.** Why does `#if 0` succeed at disabling code that `/* */` can’t?

2.11 Header files

TIME 20 min • BUILDS ON 2.7, 2.8, 2.10 • where the chapter converges

IN ONE BREATH *A header file is the bundle of declarations that other files `#include` to learn a `.cpp`’s interface — write the forward declarations once, paste them everywhere they’re needed.*

The pain it kills. With hand-written forward declarations (2.8), every file that uses `add` repeats the prototype — and every signature change means hunting down every copy. Instead: put the declaration in `add.h`, and let the preprocessor’s paste mechanism (2.10) deliver it.

```
// add.h -- the published interface
int add(int x, int y);           // declarations only. no definitions.

// add.cpp -- the implementation
#include "add.h"                 // best practice: include your own header
int add(int x, int y) { return x + y; }

// main.cpp -- a consumer
#include "add.h"                 // paste the promise; compiler satisfied
#include <iostream>
int main() { std::cout << add(4, 5); return 0; }
```



Why `add.cpp` includes its own header. It lets the compiler compare the declaration against the definition *inside one TU* — a signature mismatch becomes a cheap, precise compile error instead of a baffling link error later. Fail fast, fail in the earliest phase possible.

The rules that keep headers safe

- **Declarations in headers; definitions in `.cpp` files.** A (non-inline) function *definition* in a header gets pasted into every including TU → multiple payments for one promise → ODR part 2 → **link error**. Headers promise; source files pay.
- **Never `#include` a `.cpp` file.** Same disease, plus build-system confusion.
- **"quotes" vs. `<angle brackets>`:** quotes = your headers — searched from the including file’s directory first, then system paths; angles = headers you didn’t write (standard library, third-party) — system paths only. Note `<iostream>` has no `.h`: standard-library headers are extensionless by convention; yours use `.h`.

- **Include what you use.** If your file mentions `std::cout`, it includes `<iostream>` itself — never lean on a header that happens to drag it in (**transitive include**); that dependency can vanish in any refactor of someone else's file.
- **No using namespace in headers** — you'd force the import on every consumer forever (2.9's fuse, wholesaled).
- **Every header gets a header guard** — next section.

BRIDGE • The header IS your library/app boundary

You've been separating library layer from app layer by discipline; the header makes it *physical*. `add.h` is the menu; `add.cpp` is the kitchen; consumers compile against the menu alone. When you refactor the order validator into a proper library, its `.h` is the entire public API — everything not declared there is implementation detail you're free to gut. Bonus insight for the latency-obsessed: edit a `.cpp`, recompile one TU; edit a *header*, recompile *every TU that includes it*. Fat, over-included headers are why huge C++ codebases build slowly.

QUICK CHECK • 2.11 • answers at the back

Q1. Why does putting `int add(int, int) { ... }` (a definition) in `add.h` break a two-consumer build — and at which phase? **Q2.** What does `add.cpp` gain by including `add.h`? **Q3.** `"myfile.h"` vs. `<iostream>` — what does the punctuation tell the preprocessor?

2.12 Header guards

TIME 12 min • BUILDS ON 2.10, 2.11

IN ONE BREATH *A header guard makes a header's content survive at most one paste per translation unit — killing the duplicate-definition error caused by the same header arriving twice through different include paths.*

The problem. Includes are transitive, so diamonds happen without anyone being careless: `main.cpp` includes `a.h` and `b.h`; both include `shapes.h`. Result: `shapes.h`'s text lands in `main`'s TU *twice*. If that header contains anything that may only be defined once per TU (soon: type definitions), ODR part 1 fires.

DEFINITION • The guard pattern

```
#ifndef ADD_H      // "if ADD_H is not defined..." (first arrival: true)
#define ADD_H      // "...define it, then keep the contents"

int add(int x, int y);

#endif            // second arrival in the same TU: ADD_H exists,
                  // #ifndef is false, contents are skipped as text
```

Convention: guard name in ALL_CAPS, derived from the filename (`ADD_H`); in larger projects prefix with project/path (`MYLIB_MATH_ADD_H`) — two same-named files with the same guard silently shadow each other.

PRECISION • What guards do — and the thing they deliberately don't

Guards prevent double inclusion **within one TU**. They do *not* — and must not — prevent two *different* TUs from each getting a copy: that's the entire delivery mechanism for declarations (2.11's diagram). Consequence worth engraving: a function *definition* in a header still breaks the build **with guards in place** — each TU keeps exactly one copy, so two TUs still yield two definitions program-wide. **When each bites:** missing guard + double include → **compile-time** redefinition (one TU, ODR 1). Guarded header carrying a definition, included by two TUs → compiles clean, **link-time** multiple definition (ODR 2). Same-looking mistake, different phase — the phase tells you which rule you broke.

#pragma once. One line, same effect, auto-named. Not part of the C++ standard, but supported essentially everywhere; classic `#ifndef` guards remain the fully portable answer and are what `learncpp` standardizes on. Know both, pick one per project, be consistent.

QUICK CHECK • 2.12 • answers at the back

Q1. Walk the guard mechanism: what happens textually on the first paste vs. the second, within one TU? **Q2.** Guarded `util.h` contains a full function definition; `a.cpp` and `b.cpp` both include it. Compile outcome? Link outcome? Which ODR part? **Q3.** Why do big projects prefix guard names with the project or path?

2.13 How to design your first programs

TIME 15 min • BUILDS ON everything above

IN ONE BREATH *Design before typing: state the goal, collect constraints, decompose the task top-down until each piece is trivially small, map pieces to functions, then implement one piece at a time — testing as you go.*

Design (at the desk, not the keyboard)

- Step 1. Define the goal** — one or two sentences, stated as a user-visible outcome.
- Step 2. Collect requirements** — the constraints the solution must live inside (inputs, outputs, correctness rules, platforms) and the ways it must *not* fail.
- Step 3. Define tools and targets** — what you're building on/with, and how you'll verify it works.
- Step 4. Decompose top-down** — split the task into sub-tasks, and sub-tasks again, until each leaf is a single trivially-describable job. (The reverse — bottom-up clustering of known small steps — is also legal; real design ping-pongs between the two.)
- Step 5. Sequence the events** — order the leaves into the timeline `main` will conduct.

Implementation (now the keyboard)

- Step 1. Outline `main`** as calls to functions that don't exist yet — the leaves become names.
- Step 2. Write each function:** pin the signature first (name, parameter types, return type), then the logic, then *test it in isolation* before moving on.
- Step 3. Grow one feature at a time.** Working v1 with one feature beats a broken v3 with five. Add, test, repeat.

KEY IDEA • Write the table of contents before the chapters

Top-down design is authoring the table of contents first: if the TOC is right, each chapter is small and independently writable; if you skip it, you're writing one infinite run-on chapter and discovering the structure by archaeology.

BRIDGE • You already run this algorithm

Steps 2 and 4 are constraint-first thinking with a formal name: the problem's features corner you into the decomposition, exactly the way constraints cornered you into data structures. Step 3-of-implementation is "make it work" as doctrine — learncpp explicitly blesses shipping the crude version first. And your open grade-ledger question sits at Step 2: identifying the invariant — what's raw, what's derived — *is* requirement collection. Run the ledger through these seven steps before writing a line.

GOTCHA • Gold-plating v1

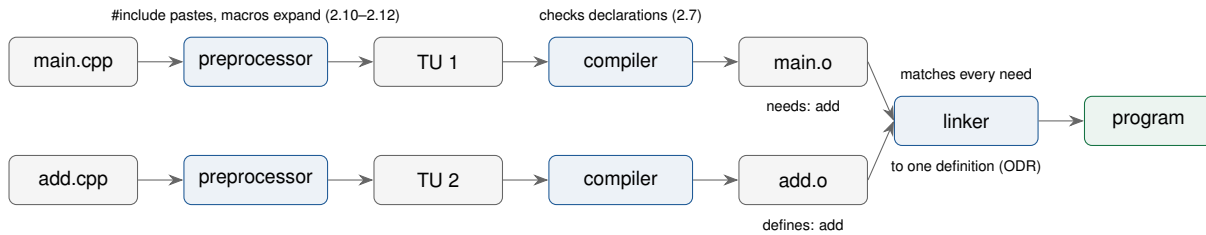
Designing for every future feature up front — **When it bites:** runtime of *your life*: weeks pass, nothing runs, motivation dies. Keep early programs modest; you *will* restructure as you learn, and that's the process working, not failing.

QUICK CHECK • 2.13 • answers at the back

Q1. Order these: write `main`'s outline / define the goal / test a function in isolation / decompose into sub-tasks / collect requirements. **Q2.** Why pin a function's *signature* before its logic?

The Big Picture: one diagram, one triage table

The whole chapter is this pipeline. Sections 2.7–2.12 are just labels on its arrows:



Error triage: read the phase, know the disease

Symptom (typical wording)	Phase	Root cause	First fix
"add was not declared in this scope"	compile	this TU never saw a declaration	#include the header / forward declare (2.7, 2.11)
"redefinition of add"	compile	two definitions inside one TU — often a header pasted twice	add the header guard; delete the duplicate (2.12)
"undefined reference to add"	link	declared and called, but no linked TU defines it	add the .cpp to the build; write the definition (2.7, 2.8)
"multiple definition of add"	link	a definition lives in a header included by 2+ TUs, or in two .cpps	move the definition to exactly one .cpp; header keeps the declaration (2.11, 2.12)

Memorize the column pairing, not the strings: **compile errors are per-file problems; link errors are program-wide bookkeeping problems.** Ask "which phase?" first, always.

One-page cheat sheet

Shapes

```
int add(int x, int y); // declaration
int add(int x, int y) // definition
{ return x + y; }
add(4, 5) // call = expression
void log(); // no value back
return; // early exit (void)
```

Guard template

```
#ifndef PROJECT_FILE_H
#define PROJECT_FILE_H
/* declarations only */
#endif
```

Word pairs (say the right one)

- **parameter** in the header / **argument** at the call
- **declaration** satisfies compiler / **definition** satisfies linker
- **scope**: compile-time, of names / **lifetime**: runtime, of objects
- frame is **popped**, not wiped
- missing return = **UB**, not “runtime error”

ODR in three lines

- one definition per TU (break it → compile error)
- one definition per program, if used (0 → undefined ref; 2+ → multiple def; both at link)
- types/templates/inline: identical copies allowed across TUs

Header rules

- headers declare; .cpp files define
- every header wears a guard
- .cpp includes its own header, first
- include what you use; never trust transitive includes
- "mine.h" vs. <not-mine>
- no using namespace in headers; never include a .cpp

Design loop (2.13) goal → requirements → decompose top-down → sequence → outline main → one function at a time: signature, logic, test → one feature at a time.

Drill bank — 15 drills, one sitting

Your standard format: one at a time, no peeking ahead. Types: PREDICT (say the outcome before running), SPOT (find the flaw), FIX (repair it), BUILD (write it), DESIGN (no code — decomposition only). Compile the PREDICT ones afterward and check yourself against reality.

D1. [BUILD • 2.7] Given `double average(double a, double b) { return (a+b)/2; }`, write its forward declaration two ways (with and without parameter names).

D2. [PREDICT • 2.2]

```
int f() { if (true) return 3; }
int g() { if (false) return 3; }
```

For each: compiles? behavior when called? Name the exact term for what's wrong with `g`.

D3. [PREDICT • 2.4]

```
void bump(int v) { v = v + 1; std::cout << v << ' '; }
int main() { int v { 7 }; bump(v); std::cout << v; }
```

Exact output, then the one-sentence mechanism.

D4. [SPOT • 2.1]

```
int main() { int helper() { return 1; } return helper(); }
```

What's illegal, and which phase objects?

D5. [PREDICT • 2.5] Two functions each define `int total{0}` and one calls the other. Does the callee's `total` disturb the caller's? Answer in stack-frame vocabulary.

D6. [PREDICT • 2.7/2.8] `main.cpp` forward-declares `int cube(int);` and calls it. No file defines `cube`. Which phase fails, and what does the message roughly say? Now delete the *call* but keep the declaration — does the build pass?

D7. [FIX • 2.8] You wrote `stats.cpp` but the build says *undefined reference to computeMean*. `main.cpp` has the correct declaration. Name the two most likely causes and the fix for each.

D8. [PREDICT • 2.12] `util.h` has *no* guard and contains only the declaration `int twice(int);`. `main.cpp` includes it twice in a row. Compile error or not — and why is the answer different if `util.h` instead held a full definition?

D9. [SPOT • 2.11/2.12] A properly guarded `shape.h` contains

```
int area(int w, int h) { return w*h; }
```

and is included by `a.cpp` and `b.cpp`. Each file compiles clean. What happens next, which ODR part, and the one-line fix.

D10. [FIX • 2.9] Two teammates' files both define `void printReport()` at file level; the linker objects. Show the namespace fix, and how each call site addresses its version.

D11. [PREDICT • 2.10]

```
#define COUNT 5
int main() { int COUNT2 { COUNT }; std::cout << COUNT2; }
```

What text does the *compiler* receive for the initializer, and does `COUNT` in `other.cpp` mean any-

thing?

- D12. [BUILD • 2.10] Using only preprocessor directives, make a program print "debug on" when a flag `DEBUG_MODE` is defined and "debug off" otherwise — flipping one line switches it.
- D13. [BUILD • 2.11/2.12] The chapter's capstone, from memory: `io.h` (guarded) declaring `readNumber` and `writeAnswer`; `io.cpp` defining them; `main.cpp` reading two ints and printing their sum. Wire it into your `new_cpp` scaffold and build it as a two-TU program.
- D14. [DESIGN • 2.13] No code. Decompose a *trade-fee calculator* (input: side, quantity, price; output: fee under a maker/taker schedule) into function *signatures only* — names, parameter types, return types. Mark which functions are queries and which are commands, and which belong to the library layer vs. the app layer.
- D15. [SPOT • whole chapter] A teammate "fixes" a multiple-definition link error by wrapping the offending header in a *second* header guard. Why does this fix nothing, and what misunderstanding about guards does it reveal?

Quick Check answers

2.1 — Q1: `main` is the caller, `greet` the callee. **Q2:** No — nested function *definitions* are illegal; compile error. **Q3:** `main` pauses; control jumps into `greet`, runs it, and resumes at the statement after the call.

2.2 — Q1: No return executes → undefined behavior; you find out (maybe) at runtime as garbage — it still compiles. **Q2:** `return` ends the function immediately, so the second statement is unreachable dead code. **Q3:** `main`'s return is the program's status code (`0/EXIT_SUCCESS` = success), and `main` alone may omit its return (implicit `return 0;`).

2.3 — Q1: Both are *compile-time* errors: a void function may not return a value; a void call yields no value to initialize `x`. **Q2:** As an *early* return — exiting before the end of the body, e.g. when a precondition fails; at the very end it's redundant.

2.4 — Q1: Parameters: `a, b`. Arguments: `width, 3`. **Q2:** Because it's perfectly legal C++ — modifying your own local copy is well-defined; only the *intent* is wrong, and compilers don't check intent. **Q3:** `g(1)` runs first; its return value becomes `f`'s first argument.

2.5 — Q1: Different objects (each lives in its own function's frame at runtime) *and* independently legal names (each has its own block scope at compile time) — both axes separate them. **Q2:** The frame is *popped*: the stack pointer moves past those bytes; nothing is erased — the values may linger until overwritten. **Q3:** Scope. Lifetime violations are runtime territory (and typically UB, not diagnosed).

2.6 — Q1: Organization, reusability, testing, extensibility, abstraction. **Q2:** DRY doesn't apply — but organization (readable `main`), testability (validate/compute in isolation), extensibility, and abstraction all still do. Repetition is only *one* of five reasons.

2.7 — Q1: Pure declaration / definition (hence also a declaration) / definition (hence also a declaration). **Q2:** Link phase; key phrase: *undefined reference to* (or *unresolved external symbol*). **Q3:** ODR part 1; compile-time redefinition error.

2.8 — Q1: TUs compile in isolation; the compiler working on `main.cpp` has zero knowledge of any other file, so the promise must appear in `main.cpp`'s own text. **Q2:** Undefined reference at *link* time — `helper.cpp` was never compiled into a linked object file. **Q3:** Symbols it *defines* and symbols it *needs*.

2.9 — Q1: Same TU: compiler, at compile time. Different TUs: linker, at link time. **Q2:** “The identifier `cout` that lives inside namespace `std`.” **Q3:** It imports every `std` name — including names *added in future library versions* — so a later toolchain can introduce a collision with your identifiers.

2.10 — Q1: Nothing of the directive itself — it's replaced by the entire (preprocessed) text of the `iostream` header. **Q2:** No. Directives are resolved per TU, top to bottom; `b.cpp` is preprocessed independently and never sees `a.cpp`'s defines. **Q3:** `#if 0` removes text before compilation and nests `fine`, while `/* */` comments can't contain another `*/`.

2.11 — Q1: Each consumer TU receives a pasted *definition*; two TUs → two program-wide definitions → ODR part 2 → *link-time* multiple-definition error. **Q2:** The compiler cross-checks declaration vs. definition within one TU, turning signature drift into an early compile error. **Q3:** Quotes: search the includer's directory first (my header); angles: system/library search paths only (not mine).

2.12 — Q1: First paste: `ADD_H` undefined → condition true → define it, keep contents. Second paste, same TU: `ADD_H` defined → condition false → contents skipped. **Q2:** Compiles clean (one copy per TU); link fails with multiple definition — ODR part 2. **Q3:** Two files named alike (e.g., two `util.h`) would share a guard name, and whichever arrives second gets silently skipped.

2.13 — Q1: Define the goal → collect requirements → decompose → outline `main` → test each function in isolation. **Q2:** The signature is the *contract* — callers (and your `main` outline) can be written and type-checked against it before the internals exist; logic can churn freely behind a stable interface.

End of Chapter 2. Next: run drills D13 and D14, tick the map, then Chapter 3 (debugging) — where the triage table starts paying rent.